🎉 Final Standards Track: ERC

ERC-4337: Account Abstraction Using Alt Mempool 💬 ‹›

Account abstraction without consensus-layer protocol changes, instead relying on higher-layer infrastructure.

Authors	Vitalik Buterin (@vbuterin), Yoav Weiss (@yoavw), Dror Tirosh (@drortirosh), Shahaf Nacson (@shahafn), Alex Forshtat (@forshtat), Kristof Gazso (@kristofgazso), Tjaden Hess (@tjade273)

Created	2021-09-29
Requires	EIP-712, EIP-7702

Table of Contents

- Abstract
- Motivation
- Specification
    - Definitions
    - The UserOperation structure
    - EntryPoint interface
    - Smart Contract Account Interface
    - Semi-abstracted Nonce Support
    - Required EntryPoint contract functionality
    - JSON-RPC API for ERC-4337
    - Support for EIP-712 signatures
    - Support for EIP-7702 authorizations
    - Extension: paymasters
    - Bundler behavior upon receiving a UserOperation
    - UserOperation Simulation
    - Estimating preVerificationGas
    - Alternative Mempools
    - Bundling
    - Error codes.
    - Factory contracts
    - Paymasters contracts
    - Aggregator contracts
    - EIP-7702 delegated Smart Contract Accounts
    - Smart Contract Accounts
    - Transient Storage

- Rationale
    - Validation Rules Rationale
    - Reputation Rationale
    - Paymasters
    - First-time Smart Contract Account creation
- Backwards Compatibility
- Security Considerations
- Copyright

Abstract

Historically, users could interact with Ethereum only by sending transactions from special accounts controlled by private keys, with transaction validation entirely enforced by fixed protocol rules. Account abstraction is an alternative model that allows each account to supply its own validation logic executed as smart contract code, while the protocol provides only minimal constraints.

This document is an account abstraction proposal which completely avoids the need for consensus-layer protocol changes. Instead of adding new protocol features and changing the bottom-layer transaction type, this proposal instead introduces a higher-layer pseudo-transaction object called a `UserOperation`. Users send `UserOperation` objects into a separate mempool. A special class of actor called bundlers package up a set of these objects into a transaction making a `handleOps` call to a special contract, and that transaction then gets included in a block.

Motivation

Historically, introducing Account Abstraction has been a long-standing goal of the Ethereum protocol. A number of proposals have been thoroughly discussed, but so far none of them have been implemented in the protocol.

This proposal takes a different approach, avoiding any adjustments to the consensus layer. It seeks to achieve the following goals:

- **Achieve the key goal of Account Abstraction**: allow users to use Smart Contract Accounts containing arbitrary verification logic instead of EOAs as their primary account. Completely remove any need at all for users to also have EOAs, as required by both status quo Smart Contract Accounts and EIP-7702.
- **Decentralization**
    - Allow any bundler (think: block builder) to participate in the process of including account-abstracted `UserOperations`
    - Work with all activity happening over a public mempool; users do not need to know the direct communication addresses (eg. IP, onion) of any specific actors
    - Avoid trust assumptions on bundlers

- **Do not require any Ethereum consensus changes**: Ethereum consensus layer development is focusing on scalability-oriented features, and there may not be any opportunity for further protocol changes for a long time. Hence, to increase the chance of faster adoption, this proposal avoids Ethereum consensus changes.
- **Support other use cases**
    - Privacy-preserving applications
    - Atomic multi-operations (similar goal to EIP-7702)
    - Pay tx fees with ERC-20 tokens, allow developers to pay fees for their users, and EIP-7702-like **sponsored transaction** use cases more generally
    - abstracting the validation allows the contract to use different signature schemes, multisig configuration, custom recovery, and more.
    - abstracting gas payments allows easy onboarding by 3rd party payments, paying with tokens, cross-chain gas payments
    - abstracting execution allows bundled transactions

Specification

Definitions

- **UserOperation** - a structure that describes a transaction to be sent on behalf of a user. To avoid confusion, it is not named "transaction".
    - Like a transaction, it contains `to`, `calldata`, `maxFeePerGas`, `maxPriorityFeePerGas`, `nonce`, `signature`.
    - Unlike a transaction, it contains several other fields, described below.
    - Notably, the `signature` field usage is not defined by the protocol, but by the Smart Contract Account implementation.
- **Sender** - the Smart Contract Account sending a `UserOperation`.
- **EntryPoint** - a singleton contract to execute bundles of `UserOperations`. Bundlers should whitelist the supported `EntryPoint`.
- **Bundler** - a node (block builder) that can handle `UserOperations`, create a valid `entryPoint.handleOps()` transaction, and add it to the block while it is still valid. This can be achieved by a number of ways:
    - Bundler can act as a block builder itself.
    - If the bundler is not a block builder, it should work with the block builder through an infrastructure such as `mev-boost`, or any other kind of proposer-builder separation.
- **Paymaster** - a helper contract that agrees to pay for the transaction, instead of the sender itself.
- **Factory** - a helper contract that performs a deployment for a new `sender` contract if necessary.

- **Aggregator** - also known as “authorizer contract” - a contract that enables multiple `UserOperations` to share a single validation. The full design of such contracts is outside the scope of this proposal.
- **Canonical** `UserOperation` **mempool** - a decentralized permissionless P2P network where bundlers exchange `UserOperations` that are valid and conform with the same shared set of rules applied to the validation code. The full specification of such rules is outside the scope of this proposal.
- **Alternative** `UserOperation` **mempool** - any other P2P mempool where the validity of `UserOperations` is determined by rules that are different from the shared set of rules, applied to the validation code, in any way.
- **Deposit** - an amount of Ether (or any L2 native currency) that a `Sender` or `Paymaster` contract has transferred to the `EntryPoint` contract intended to pay gas costs of the future `UserOperations`.

The `UserOperation` structure

To avoid Ethereum consensus changes, we do not attempt to create new transaction types for account-abstracted transactions. Instead, users package up the action they want their Smart Contract Account to take in a struct named `UserOperation`:

Field	Type	Description
`sender`	`address`	The Account making the `UserOperation`
`nonce`	`uint256`	Anti-replay parameter (see “Semi-abstracted Nonce Support”)
`factory`	`address`	Account Factory for new Accounts OR `0x7702` flag for EIP-7702 Accounts, otherwise `address(0)`
`factoryData`	`bytes`	data for the Account Factory if `factory` is provided OR EIP-7702 initialization data, or empty array
`callData`	`bytes`	The data to pass to the `sender` during the main execution call
`callGasLimit`	`uint256`	The amount of gas to allocate the main execution call
`verificationGasLimit`	`uint256`	The amount of gas to allocate for the verification step
`preVerificationGas`	`uint256`	Extra gas to pay the bundler
`maxFeePerGas`	`uint256`	Maximum fee per gas (similar to EIP-1559 `max_fee_per_gas`)
`maxPriorityFeePerGas`	`uint256`	Maximum priority fee per gas (similar to EIP-1559 `max_priority_fee_per_gas`)
`paymaster`	`address`	Address of paymaster contract, (or empty, if the `sender` pays for gas by itself)
`paymasterVerificationGasLimit`	`uint256`	The amount of gas to allocate for the paymaster validation code (only if paymaster exists)

`paymasterPostOpGasLimit`	`uint256`	The amount of gas to allocate for the paymaster post-operation code (only if paymaster exists)
`paymasterData`	`bytes`	Data for paymaster (only if paymaster exists)
`signature`	`bytes`	Data passed into the `sender` to verify authorization

Users send `UserOperation` objects to a dedicated `UserOperation` mempool.

To prevent replay attacks, either cross-chain or with multiple `EntryPoint` contract versions, the `signature` MUST depend on `chainid` and the `EntryPoint` address.

Note that one EIP-7702 “authorization tuple” value can be provided alongside the `UserOperation` struct, but “authorization tuples” are not included in the `UserOperation` itself.

`EntryPoint` interface

When passed on-chain, to the `EntryPoint` contract, the `Account` and the `Paymaster`, a “packed” version of the above structure called `PackedUserOperation` is used:

Field	Type	Description
`sender`	`address`	
`nonce`	`uint256`	
`initCode`	`bytes`	concatenation of factory address and factoryData (or empty), or EIP-7702 data
`callData`	`bytes`	
`accountGasLimits`	`bytes32`	concatenation of verificationGasLimit (16 bytes) and callGasLimit (16 bytes)
`preVerificationGas`	`uint256`	
`gasFees`	`bytes32`	concatenation of maxPriorityFeePerGas (16 bytes) and maxFeePerGas (16 bytes)
`paymasterAndData`	`bytes`	concatenation of paymaster fields (or empty)
`signature`	`bytes`	

The core interface of the `EntryPoint` contract is as follows:

```
function handleOps(PackedUserOperation[] calldata ops, address payable benefi
```

The `beneficiary` is the address that will be paid with all the gas fees collected during the execution of the bundle.

Smart Contract Account Interface

The core interface required for the Smart Contract Account to have is:

```
interface IAccount {
  function validateUserOp
      (PackedUserOperation calldata userOp, bytes32 userOpHash, uint256 missi
      external returns (uint256 validationData);
}
```

The `userOpHash` is a hash over the `userOp` (except `signature`), `entryPoint` and `chainId`.

The Smart Contract Account:

- MUST validate the caller is a trusted `EntryPoint`
- MUST validate that the signature is a valid signature of the `userOpHash`, and SHOULD return `SIG_VALIDATION_FAILED` (`1`) without reverting on signature mismatch. Any other error MUST revert.
- SHOULD not return early when returning `SIG_VALIDATION_FAILED` (`1`). Instead, it SHOULD complete the normal flow to enable performing a gas estimation for the validation function.
- MUST pay the `EntryPoint` (caller) at least the `missingAccountFunds` (which might be zero, in case the current `sender`'s deposit is sufficient)
- The `sender` MAY pay more than this minimum to cover future transactions. It can also call `withdrawTo` to retrieve it later at any time.
- The return value MUST be packed of `aggregator`/`authorizer`, `validUntil` and `validAfter` timestamps.
    - `aggregator`/`authorizer` - 0 for valid signature, 1 to mark signature failure. Otherwise, an address of an `aggregator`/`authorizer` contract.
    - `validUntil` is 6-byte timestamp value, or zero for "infinite". The `UserOperation` is valid only up to this time.
    - `validAfter` is 6-byte timestamp. The `UserOperation` is valid only after this time.
    - In order to specify a validity range using block numbers, both the `validUntil` and `validAfter` need to set their highest bit to 1.
    - **Note:** The validity range can be expressed by two block timestamps or two block numbers, but one timestamp and one block number cannot be mixed in the same UserOperation's validity range.

The Smart Contract Account MAY implement the interface `IAccountExecute`

```
interface IAccountExecute {
  function executeUserOp(PackedUserOperation calldata userOp, bytes32 userOpH
}
```

This method will be called by the `EntryPoint` with the current UserOperation, instead of executing the `callData` itself directly on the `sender`.

Semi-abstracted Nonce Support

In Ethereum protocol, the sequential transaction `nonce` value is used as a replay protection method as well as to determine the valid order of transaction being included in blocks.

It also contributes to the transaction hash uniqueness, as a transaction by the same sender with the same nonce may not be included in the chain twice.

However, requiring a single sequential `nonce` value is limiting to the senders' ability to define their custom logic with regard to transaction ordering and replay protection.

Instead of sequential `nonce` we implement a nonce mechanism that uses a single `uint256` nonce value in the `UserOperation`, but treats it as two values:

- 192-bit "key"
- 64-bit "sequence"

These values are represented on-chain in the `EntryPoint` contract. We define the following method in the `EntryPoint` interface to expose these values:

```
function getNonce(address sender, uint192 key) external view returns (uint256
```

For each `key` the `sequence` is validated by the `EntryPoint` for each UserOperation. If the nonce validation fails the `UserOperation` is considered invalid and the bundle is reverted. The `sequence` value is incremented sequentially and monotonically for the `sender` for each UserOperation. A new `key` can be introduced with an arbitrary value at any point, with its `sequence` starting at `0`.

This approach maintains the guarantee of `UserOperation` hash uniqueness on-chain on the protocol level while allowing Accounts to implement any custom logic they may need operating on a 192-bit "key" field, while fitting the 32 byte word.

Reading and validating the nonce

When preparing the `UserOperation` bundlers may make a view call to this method to determine a valid value for the `nonce` field.

Bundler's validation of a `UserOperation` SHOULD start with `getNonce` to ensure the transaction has a valid `nonce` field.

If the bundler is willing to accept multiple `UserOperations` by the same sender into their mempool, this bundler is supposed to track the `key` and `sequence` pair of the `UserOperations` already added in the mempool.

Usage examples

1. Classic sequential nonce.

   In order to require the Account to have classic, sequential nonce, the validation function MUST perform:

   ```
   require(userOp.nonce<type(uint64).max)
   ```

2. Ordered administrative events

In some cases, an account may need to have an “administrative” channel of operations running in parallel to normal operations.

In this case, the account may use a specific `key` when calling methods on the account itself:

```
bytes4 sig = bytes4(userOp.callData[0 : 4]);
uint key = userOp.nonce >> 64;
if (sig == ADMIN_METHODSIG) {
    require(key == ADMIN_KEY, "wrong nonce-key for admin operation"
} else {
    require(key == 0, "wrong nonce-key for normal operation");
}
```

Required `EntryPoint` contract functionality

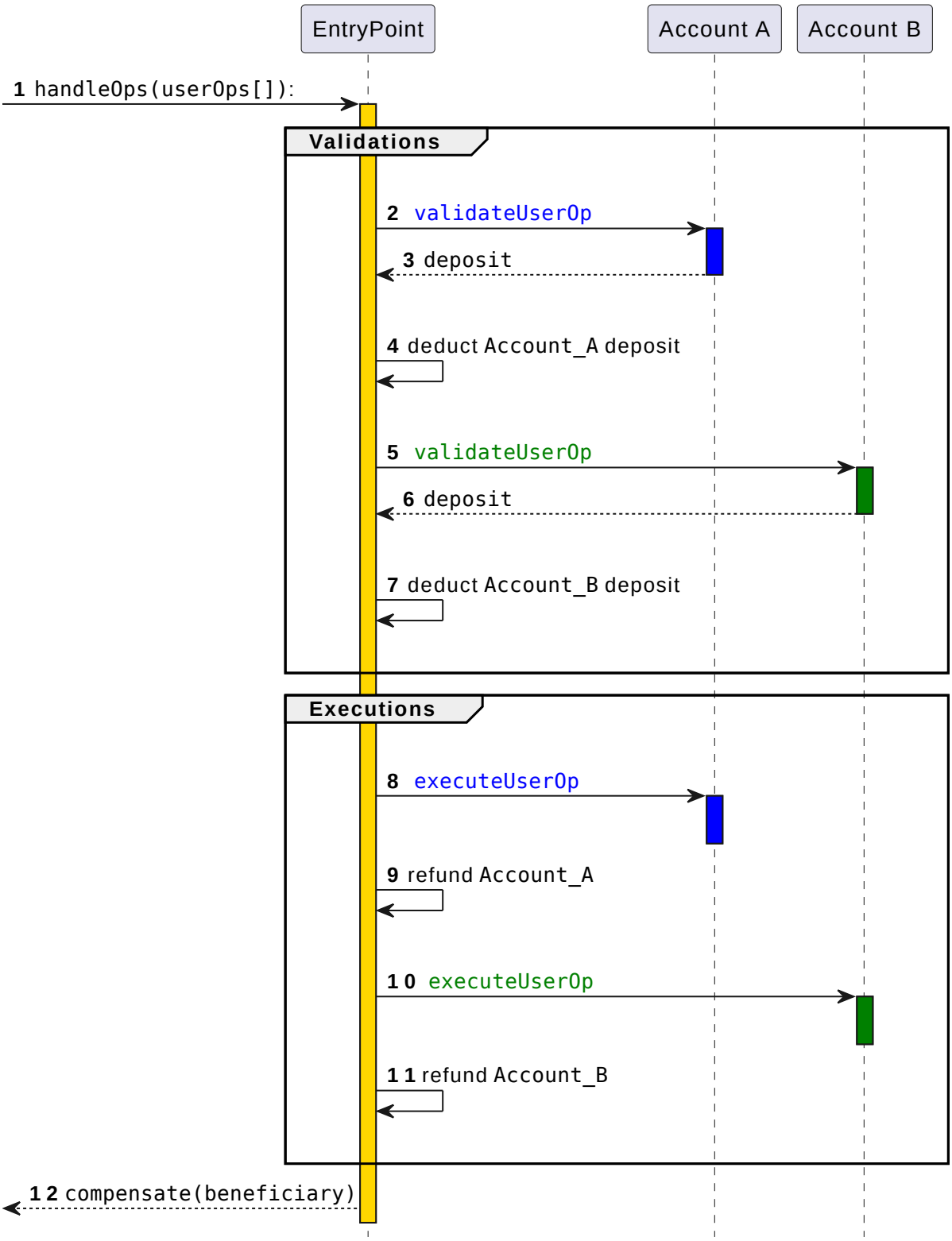
The `EntryPoint` method is `handleOps`, which handles an array of `UserOperations`

The `EntryPoint`’s `handleOps` function must perform the following steps (we first describe the simpler non-paymaster case). It must make two loops, the **verification loop** and the **execution loop**. In the verification loop, the `handleOps` call must perform the following steps for each `UserOperation`:

- **Create the `sender` Smart Contract Account if it does not yet exist**, using the `initcode` provided in the `UserOperation`.
    - If the `factory` address is “0x7702”, then the sender MUST be an EOA with an EIP-7702 authorization designation. The `EntryPoint` validates the authorized address matches the one specified in the `UserOperation` signature (see
      Support for [EIP-7702] authorizations).
    - If the `sender` does not exist, *and* the `initcode` is empty, or does not deploy a contract at the “sender” address, the call must fail.
    - **WARNING**: If the `sender` does exist, *and* the `initcode` is *not* empty, then the `initcode` is ignored.
- calculate the maximum possible fee the `sender` needs to pay based on validation and call gas limits, and current gas values.
- calculate the fee the `sender` must add to its “deposit” in the `EntryPoint`
- **Call `validateUserOp` on the `sender` contract**, passing in the `UserOperation`, its hash and the required fee. The Smart Contract Account MUST verify the `UserOperation`’s `signature` parameter, and pay the fee if the `sender` considers the `UserOperation` valid. If any `validateUserOp` call fails, `handleOps` must skip execution of at least that `UserOperation`, and may revert entirely.
- Validate the account’s deposit in the `EntryPoint` is high enough to cover the max possible cost (cover the already-done verification and max execution gas)

In the execution loop, the `handleOps` call must perform the following steps for each `UserOperation`:

- **Call the account with the `UserOperation`'s calldata**. It's up to the account to choose how to parse the calldata; an expected workflow is for the account to have an `execute` function that parses the remaining calldata as a series of one or more calls that the account should make.
- If the calldata starts with the methodsig `IAccountExecute.executeUserOp`, then the `EntryPoint` must build a calldata by encoding `executeUserOp(userOp,userOpHash)` and call the account using that calldata.
- After the call, refund the account's deposit with the excess gas cost that was pre-charged.
  A penalty of `10%` (`UNUSED_GAS_PENALTY_PERCENT`) is applied on the amounts of `callGasLimit` and `paymasterPostOpGasLimit` gas that remains **unused**.
  This penalty is only applied if the amount of the remaining unused gas is greater than or equal `40000` (`PENALTY_GAS_THRESHOLD`).
  This penalty is necessary to prevent the `UserOperations` from reserving large parts of the gas space in the bundle but leaving it unused and preventing the bundler from including other `UserOperations`.
- After the execution of all calls, pay the collected fees from all `UserOperations` to the `beneficiary` address provided by the bundler.



Before accepting a `UserOperation`, bundlers SHOULD use an RPC method to locally call the `handleOps` function on the `EntryPoint`, to verify that the signature is correct and the `UserOperation` actually pays fees; see the Simulation section below for details. A node/bundler MUST reject a `UserOperation` that fails the validation, meaning not adding it to the local mempool and not propagating it to other peers.

JSON-RPC API for ERC-4337

In order to support sending `UserOperation` objects to bundlers, which in turn propagate them through the P2P mempool, we introduce a set of JSON-RPC APIs including

`eth_sendUserOperation` and `eth_getUserOperationReceipt`.

The full definition of the new JSON-RPC API is outside the scope of this proposal.

Support for EIP-712 signatures

The `userOpHash` is calculated as an [EIP-712] typed message hash with the following parameters:

```
bytes32 constant TYPE_HASH =
    keccak256(
        "EIP712Domain(string name,string version,uint256 chainId,address veri
    );

bytes32 constant PACKED_USEROP_TYPEHASH =
    keccak256(
        "PackedUserOperation(address sender,uint256 nonce,bytes initCode,byte
    );
```

Support for EIP-7702 authorizations

On networks with EIP-7702 enabled, the `eth_sendUserOperation` method accepts an extra `eip7702Auth` parameter. If this parameter is set, it MUST be a valid EIP-7702 authorization tuple, and signed by the `sender` address. The bundler MUST add all required `eip7702Auth` of all `UserOperations` in a bundle to the `authorizationList` and execute the bundle using a transaction type `SET_CODE_TX_TYPE`. Additionally, the `UserOperation` hash calculation is updated to include the desired EIP-7702 delegation address.

If the `initCode` field starts with `0x7702` right-padded with 18 zeros, and this account was deployed using an EIP-7702 transaction, then the hash is calculated as follows:

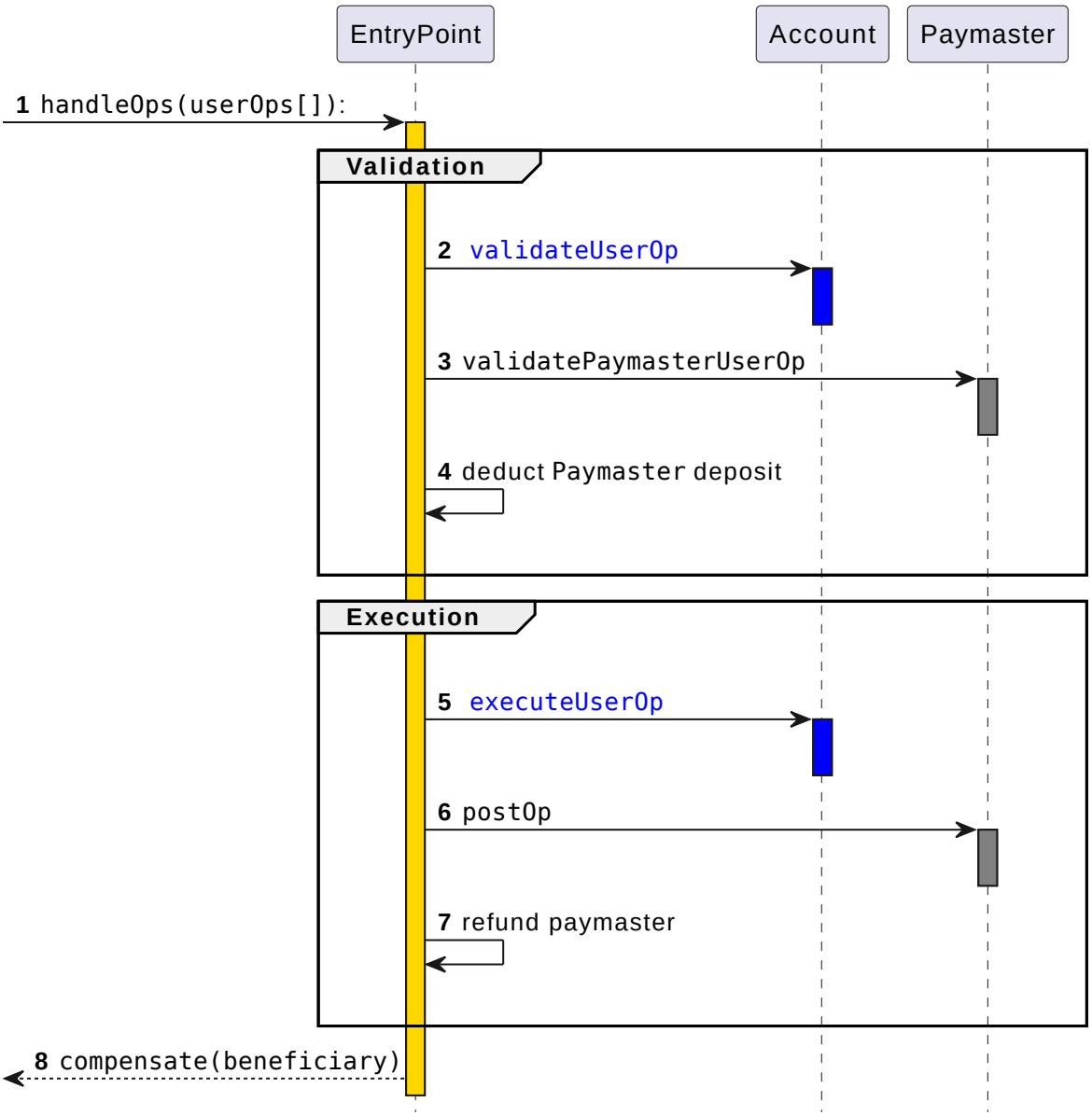
- For the purpose of hash calculation, the first 20 bytes of the `initCode` field of the `UserOperation` are set to account's EIP-7702 delegate address (fetched with EXTCODECOPY)
- The `initCode` is not used to call a factory contract.
- If the `initCode` is longer than 20 bytes, then the rest of the initCode is used to call an initialization function in the account itself.

Note that a `UserOperation` may still be executed without such `initCode`. In this case the `EntryPoint` doesn't hash the current EIP-7702 delegate, and can be potentially executed against a modified account.

Additionally, EIP-7702 defines the gas cost of executing an authorization equal to `PER_EMPTY_ACCOUNT_COST = 25000`. This gas consumption is not observable on-chain by the `EntryPoint` contract and MUST be included in the `preVerificationGas` value.

Extension: paymasters

We extend the `EntryPoint` logic to support **paymasters** that can sponsor transactions for other users. This feature can be used to allow application developers to subsidize fees for their users, allow users to pay fees with ERC-20 tokens and many other use cases. When the `paymasterAndData` field in the `UserOperation` is not empty, the `EntryPoint` implements a different flow for that UserOperation:



During the verification loop, in addition to calling `validateUserOp`, the `handleOps` execution also must check that the paymaster has enough ETH deposited with the `EntryPoint` to pay for the `UserOperation`, and then call `validatePaymasterUserOp` on the paymaster to verify that the paymaster is willing to pay for the `UserOperation`. Note that in this case, the `validateUserOp` is called with a `missingAccountFunds` of 0 to reflect that the account's deposit is not used for payment for this `UserOperation`.

If the paymaster's `validatePaymasterUserOp` returns a non-empty `context` byte array, then `handleOps` must call `postOp` on the paymaster after making the main execution call. Otherwise, no call is done to the `postOp` function.

Maliciously crafted paymasters could pose a risk of a DoS attack against the system and bundlers should take steps to mitigate it. As a mitigation, bundlers should use a reputation system for contracts they serve, and the paymaster must either limit its storage usage, or deposit a stake in a reputation system. Full specification of a reputation system is outside the scope of this proposal.

The `paymasterAndData` field encoding and `paymasterSignature`

The `paymasterAndData` field is a byte array that contains a non-standard encoding of the following fields:

- `paymasterAddress` - 20 bytes — the address of the paymaster contract
- `paymasterVerificationGasLimit` - 16 bytes - the gas limit for the verification function
- `postOpGasLimit` - 16 bytes - the gas limit for the postOp function
- `paymasterData` - the data that the paymaster contract will receive in the `validatePaymasterUserOp` call

The following data can optionally be appended to the `paymasterAndData` field:

- `paymasterSignature` - the "signature" value byte array to be checked by the paymaster contract; this value can be provided **without affecting the UserOperation hash**
- `paymasterSignatureLength` - 2 bytes - the exact length of the `paymasterSignature` parameter byte array
- `PAYMASTER_SIG_MAGIC` (`0x22e325a297439656`) - the magic value that is appended to indicate the use of the `paymasterSignature` feature by the UserOperation

Note that as both the `signature` and the `paymasterSignature` fields do not affect the UserOperation hash, the signing by the Sender and the Paymaster can be performed in parallel.

The paymaster interface is as follows:

```
function validatePaymasterUserOp
    (PackedUserOperation calldata userOp, bytes32 userOpHash, uint256 maxCost
    external returns (bytes memory context, uint256 validationData);

function postOp
    (PostOpMode mode, bytes calldata context, uint256 actualGasCost, uint256
    external;

enum PostOpMode {
    opSucceeded, // UserOperation succeeded
```

```
    opReverted // UserOperation reverted. paymaster still has to pay for gas.
}
```

The `EntryPoint` must implement the following API to let entities like paymasters have a stake, and thus have more flexibility in their storage access.

```
// add a stake to the calling entity
function addStake(uint32 _unstakeDelaySec) external payable;

// unlock the stake (must wait unstakeDelay before can withdraw)
function unlockStake() external;

// withdraw the unlocked stake
function withdrawStake(address payable withdrawAddress) external;
```

The paymaster must also have a deposit, which the `EntryPoint` will charge `UserOperation` costs from. The deposit (for paying gas fees) is separate from the stake (which is locked).

The `EntryPoint` must implement the following interface to allow Paymasters (and optionally Accounts) to manage their deposit:

```
// return the deposit of an account
function balanceOf(address account) public view returns (uint256);

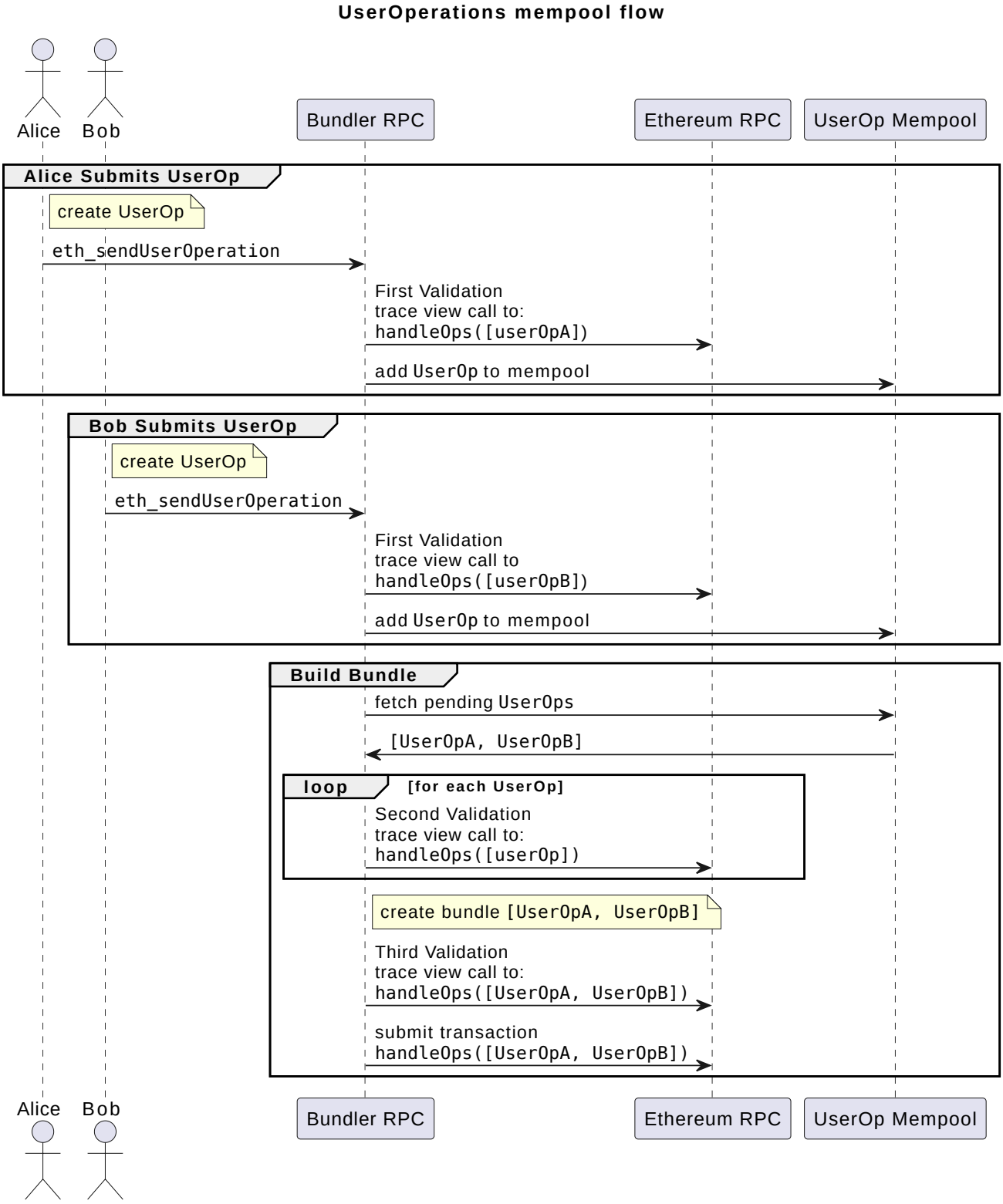
// add to the deposit of the given account
function depositTo(address account) public payable;

// add to the deposit of the calling account
receive() external payable;

// withdraw from the deposit of the current account
function withdrawTo(address payable withdrawAddress, uint256 withdrawAmount)

// get the currently executing UserOperation hash, or 0 if not called during
function getCurrentUserOpHash() public view returns (bytes32);
```

Bundler behavior upon receiving a UserOperation



Similar to an Ethereum transaction, the offchain flow of a `UserOperation` can be described as follows:

1. Client sends a `UserOperation` to the bundler through an RPC call `eth_sendUserOperation`.
2. Before including the `UserOperation` in the mempool, the bundler runs the *first validation* of the newly received UserOperation. If the `UserOperation` fails validation, the bundler drops it and returns an error in response to `eth_sendUserOperation`.

3. Later, once building a bundle, the bundler takes `UserOperations` from the mempool and runs the *second validation* of a single `UserOperation` on each of them. If it succeeds, it is scheduled for inclusion in the next bundle, and dropped otherwise.
4. Before submitting the new bundle onchain, the bundler performs the *third validation* of the entire `UserOperations` bundle. If any of the `UserOperations` fail validation, the bundler drops them. The bundler should keep track of the peers' reputation. The full design of such a reputation system is outside the scope of this proposal.

When a bundler receives a `UserOperation`, it must first run some basic sanity checks, namely that:

- Either the `sender` is an existing contract, or the `initCode` is not empty (but not both)
- If `initCode` is not empty, parse its first 20 bytes as a factory address or an EIP-7702 flag.
  Record whether the factory is staked, in case the later simulation indicates that it needs to be. If the factory accesses the global state, it must be staked.
- The `verificationGasLimit` and `paymasterVerificationGasLimits` are lower than `MAX_VERIFICATION_GAS` (`500000`) and the `preVerificationGas` is high enough to pay for the calldata gas cost of serializing the `UserOperation` plus `PRE_VERIFICATION_OVERHEAD_GAS` (`50000`).
- The `paymasterAndData` is either empty, or starts with the **paymaster** address, which is a contract that (i) currently has nonempty code on chain, (ii) has a sufficient deposit to pay for the UserOperation, and (iii) is not currently banned. During simulation, the paymaster's stake is also checked, depending on its storage usage.
- The `callGasLimit` is at least the cost of a `CALL` with non-zero value.
- The `maxFeePerGas` and `maxPriorityFeePerGas` are above a configurable minimum value that the bundler is willing to accept. At the minimum, they are sufficiently high to be included with the upcoming `block.basefee`.
- The `sender` doesn't have another `UserOperation` already present in the mempool (or it replaces an existing entry with the same sender and nonce, with a higher `maxPriorityFeePerGas` and an equally increased `maxFeePerGas`). Only one `UserOperation` per sender may be included in a single bundle. A sender is exempt from this rule and may have multiple `UserOperations` in the mempool and in a bundle if it is staked.

UserOperation Simulation

We define `UserOperation` simulation, as the offchain view call (or trace call) to the `EntryPoint` contract with the `UserOperation`, and the enforcement of the shared set of rules applied to the validation code, as part of the `UserOperation` validation.

Simulation Rationale

To validate a normal Ethereum transaction `tx`, the bundler performs static checks, like:

1. `ecrecover(tx.v, tx.r, tx.s)` has to return a valid EOA
2. `tx.nonce` has to be the current nonce of the recovered EOA
3. `balance` of the recovered EOA has to be sufficient to pay for the transaction
4. `tx.gasLimit` has to be sufficient to cover the intrinsic gas cost of a transaction
5. `chainId` has to match the current chain

All of these checks do not rely on EVM state, and cannot be affected by other Accounts' transactions.

In contrast, `UserOperation` validation rely on EVM state (calls to `validateUserOp`, `validatePaymasterUserOp`), can be changed by other `UserOperations` (or normal Ethereum transactions). Therefore, we introduce simulation as a new mechanism to check its validity. Intuitively, the aim of the simulation is to ensure the onchain validation code of a `UserOperation` is sandboxed, isolated from other `UserOperations` in the same bundle.

Simulation Specification:

To simulate a `UserOperation` validation, the bundler makes a view call to the `handleOps()` method with the `UserOperation` to check.

Simulation should run only on the validation section of the `sender` and `paymaster`, and is not required for the `UserOperation`'s execution. A bundler MAY add second "always failed" `UserOperation` to the bundle, so that the simulation will end as soon as the first UserOperation's validation complete.

The bundler MUST drop the `UserOperation` if the simulation reverts

The simulated call performs the full validation, by calling:

1. If `initCode` is present, create the `sender` Account.
2. `account.validateUserOp`.
3. if specified a paymaster: `paymaster.validatePaymasterUserOp`.

Either `sender` or `paymaster` may return a time-range (`validAfter` / `validUntil`). The `UserOperation` MUST be valid at the current time to be considered valid, defined as `validAfter<=block.timestamp`.

A bundler MUST drop a `UserOperation` if it expires too soon and is likely to become invalid before the next block. To decode the returned time-ranges, the bundler MUST run the validation using tracing, to decode the return value from the `validateUserOp` and `validatePaymasterUserOp` methods.

To prevent DoS attacks on bundlers, they must make sure the validation methods above pass the validation rules, which constrain their usage of opcodes and storage. The full

design of such a shared set of rules, applied to the validation code, is outside the scope of this proposal.

Estimating `preVerificationGas`

This document does not specify a canonical way to estimate this value, as it depends on non-permanent network properties such as operation and data gas pricing and the expected bundle size.

However, the requirement is for the estimated value to be sufficient to cover the following costs:

- Base bundle transaction cost. On Ethereum, `21000` gas divided by the number of `UserOperations`.
- The calldata gas cost related to the `UserOperation` as defined in EIP-2028.
- Static `EntryPoint` contract code execution.
- Static memory cost when loading the fixed size fields of the `UserOperation` into EVM memory
- Memory cost (including expansion cost) due to context returned by paymaster `validatePaymasterUserOp` function, if relevant.
    - External call to the `innerHandleOp()` function which is a major part of the `EntryPoint` implementation. Note that this value is not static and depends on the `UserOperation`'s position in the bundle.
- [EIP-7702] authorization cost, if any.
- EIP-7623 calldata floor price increase is estimated as follows:
    - Apply the new formula for `tx.gasUsed`, replacing the `execution_gas_used` value with an estimate for value made for this UserOperation.
    - The estimate is calculated as a sum of all verification gas used during simulation (account creation, validation and paymaster validation) and 10% of the sum of execution and `postOp` gas limit.

The bundler MUST require a slack in `PreVerificationGas` value, to accommodate memory expansion costs in the future bundle, and the expected position of the `UserOperation` in it.

Alternative Mempools

The simulation rules above are strict and prevent the ability of paymasters to grief the system. However, there might be use cases where specific paymasters can be validated (through manual auditing) and verified that they cannot cause any problem, while still require relaxing of the opcode rules. A bundler cannot simply "whitelist" a request from a specific paymaster: if that paymaster is not accepted by all bundlers, then its support will be sporadic at best. Instead, we introduce the term "alternate mempool": a modified validation rules, and procedure of propagating them to other bundlers.

The procedure of using alternate mempools is outside the scope of this proposal.

Bundling

Bundling is the process where a node/bundler collects multiple `UserOperations` and creates a single transaction to submit on-chain.

During bundling, the bundler MUST:

- Exclude `UserOperations` that access any sender address of another `UserOperation` in the same bundle.
- Exclude `UserOperations` that access any address created by another `UserOperation` validation in the same bundle (via a factory).
- For each paymaster used in the bundle, keep track of the balance while adding `UserOperations`. Ensure that it has sufficient deposit to pay for all the `UserOperations` that use it.

After creating the bundle, before including the transaction in a block, the bundler SHOULD:

- Run `debug_traceCall` with maximum possible gas, to enforce the validation rules on opcode and storage access, as well as to verify the entire `handleOps` bundle transaction, and use the consumed gas for the actual transaction execution.
- If the call reverted, the bundler MUST use the trace result to find the entity that reverted the call.
  This is the last entity that is CALL'ed by the `EntryPoint` prior to the revert.
  (the bundler cannot assume the revert is `FailedOp`)
- If any verification context rule was violated the bundlers MUST treat it the same as if this `UserOperation` reverted.
- Remove the offending `UserOperation` from the current bundle and from mempool.
- If the error is caused by a `factory` or a `paymaster`, and the `sender` of the `UserOperation` **is not** a staked entity, then issue a "ban" for the guilty factory or paymaster.
- If the error is caused by a `factory` or a `paymaster`, and the `sender` of the `UserOperation` **is** a staked entity, do not ban the `factory` / `paymaster` from the mempool. Instead, issue a "ban" for the staked `sender` entity.
- Repeat until `debug_traceCall` succeeds.

As staked entries may use some kind of transient storage to communicate data between `UserOperations` in the same bundle, it is critical that the exact same opcode and precompile banning rules as well as storage access rules are enforced for the `handleOps` validation in its entirety as for individual `UserOperations`. Otherwise, attackers may be able to use the banned opcodes to detect running on-chain and trigger a `FailedOp` revert.

When a bundler includes a bundle in a block it must ensure that earlier transactions in the block don't make any `UserOperation` fail.

Error codes.

While performing validation, the `EntryPoint` must revert on failures. During simulation, the calling bundler MUST be able to determine which entity (`sender`, `factory` or `paymaster`) caused the failure. The attribution of a revert to an entity is done using call-tracing: the last entity called by the `EntryPoint` prior to the revert is the entity that caused the revert.

- For diagnostic purposes, the `EntryPoint` must only revert with explicit `SignatureValidationFailed()`, `FailedOp()` or `FailedOpWithRevert()` errors.
- The message of the error starts with event code, AA##
- Event code starting with "AA1" signifies an error during `sender` creation
- Event code starting with "AA2" signifies an error during `sender` validation (`validateUserOp`)
- Event code starting with "AA3" signifies an error during `paymaster` validation (`validatePaymasterUserOp`)

Factory contracts

All `factory` contracts MUST check that all calls to the `createAccount()` function originate from the `entryPoint.senderCreator()` address.

Paymasters contracts

All `paymaster` contracts MUST check that all calls to the `validatePaymasterUserOp()` and `postOp()` functions originate from the `EntryPoint`.

Aggregator contracts

All `aggregator` contracts MUST check that all calls to the `validateSignatures()` function originates from the `EntryPoint`.

EIP-7702 delegated Smart Contract Accounts

All EIP-7702 delegated Smart Contract Account implementations MUST check that all calls to the initialization function originate from the `entryPoint.senderCreator()` address.

There is no way for the `EntryPoint` contract to know whether an EIP-7702 account has been initialized or not, and therefore the EIP-7702 account initialization code, can be called multiple times through `EntryPoint`. The Account code SHOULD only allow calling it once and the Wallet Application SHOULD NOT pass the `initCode` repeatedly.

Smart Contract Accounts

Storage layout collisions

It is expected that most of ERC-4337 Smart Contract Account will be upgradeable, either via on-chain delegate proxy contracts or via EIP-7702.

When changing the underlying implementation, all Accounts MUST ensure that there are no conflicts in the storage layout of the two contracts.

One common approach to this problem is often referred to as "diamond storage" and is fully described in ERC-7201.

Transient Storage

Contracts using the EIP-1153 transient storage MUST take into account that ERC-4337 allows multiple `UserOperations` from different unrelated `sender` addresses to be included in the same underlying transaction. The transient storage MUST be cleaned up manually if contains any sensitive information or is used for access control.

Rationale

The main challenge with a purely "Smart Contract Accounts" based Account Abstraction system is DoS safety: how can a block builder including an operation make sure that it will actually pay fees, without having to first execute the entire operation? Requiring the block builder to execute the entire operation opens a DoS attack vector, as an attacker could easily send many operations that pretend to pay a fee but then revert at the last moment after a long execution. Similarly, to prevent attackers from cheaply clogging the mempool, nodes in the P2P network need to check if an operation will pay a fee before they are willing to forward it.

The first step is a clean separation between validation (acceptance of UserOperation, and acceptance to pay) and execution. In this proposal, we expect Accounts to have a `validateUserOp` method that takes as input a `UserOperation`, verifies the signature and pays the fee. Only if this method returns successfully, the execution will happen.

The `EntryPoint`-based approach allows for a clean separation between verification and execution, and keeps Smart Contract Accounts' logic simple. It enforces the simple rule that only after validation is successful and the `UserOperation` can pay, the execution is done and only done once, and also guarantees the fee payment.

Validation Rules Rationale

The next step is protecting the bundlers from denial-of-service attacks by a mass number of `UserOperations` that appear to be valid (and pay) but that eventually revert, and thus block the bundler from processing valid `UserOperations`.

There are two types of `UserOperations` that can fail validation:

1. `UserOperations` that succeed in initial validation (and accepted into the mempool), but rely on the environment state to fail later when attempting to include them in a block.
2. `UserOperations` that are valid when checked independently but fail when bundled together to be put on-chain. To prevent such rogue `UserOperations`, the bundler is required to follow a set of shared set of rules applied to the validation code, to prevent such denial-of-service attacks.

Reputation Rationale

UserOperation's storage access rules prevent them from interfering with each other. But "global" entities - paymasters and factories are accessed by multiple `UserOperations`, and thus might invalidate multiple previously valid `UserOperations`.

To prevent abuse, we throttle down (or completely ban for a period of time) an entity that causes invalidation of a large number of `UserOperations` in the mempool. To prevent such entities from "Sybil-attack", we require them to stake with the system, and thus make such DoS attack very expensive. Note that this stake is never slashed. There is no slashing mechanism involved and the only use for the stake in sybil attack prevention. The stake can be withdrawn at any time after the specified unstake delay.

Unstaked entities are allowed, under the rules below.

When staked, an entity is less restricted in its use of contract storage.

The stake value is not enforced on-chain, but specifically by each bundler while simulating a transaction.

Paymasters

Paymaster contracts allow the abstraction of gas: having a contract, that is not the sender of the transaction, to pay for the transaction fees.

Paymaster architecture allows them to follow the model of "pre-charge, and later refund". E.g. a token-paymaster may pre-charge the user with the max possible price of the transaction, and refund the user with the excess afterwards.

First-time Smart Contract Account creation

NOTE: for contracts using EIP-7702 this flow is described in
Support for EIP-7702 authorizations.

It is an important design goal of this proposal to replicate the key property of EOAs that users do not need to perform some custom action or rely on an existing user to create their Smart Contract Account; they can simply generate an address locally and immediately start accepting funds.

The Smart Contract Account creation itself is done by a "factory" contract, with some Account-specific data. The Factory is expected to use `CREATE2 0xF5` (not `CREATE 0xF0`) to create the Account, so that the order of creation of the Accounts doesn't interfere with the generated addresses. The `initCode` field (if non-zero length) is parsed as a 20-byte `factory` address, followed by `calldata` to pass to this address. This method call is expected to create the Account and return its address. If the factory does use `CREATE2 0xF5` or some other deterministic method to create the Account, it's expected to return the Account address even if it had already been created. This comes to make it easier for bundlers to query the address without knowing if the Account has already been deployed, by simulating a call to `entryPoint.getSenderAddress()`, which calls the `factory` under the hood. When `initCode` is specified, if either the `sender` address points to an existing contract or the `sender` address still does not exist after calling the `initCode`, then the operation is aborted. The `initCode` is not called directly from the `EntryPoint`, but from another address. This is a security measure as calls originated from `EntryPoint` may be unconditionally trusted by wallets or paymasters. If the contract created by this factory method later accepts a call to `validateUserOp` to validate the `UserOperation`'s signature, the account creation UserOperation is successfully validated. For security reasons, it is important that the generated contract address will depend on the initial signature. This way, even if someone can deploy an Account at that address, he can't set different credentials to control it. The Factory has to be staked if it accesses global storage. NOTE: In order for the Wallet Application to determine the "counterfactual" address of the Account prior to its creation, it makes a static call to the `entryPoint.getSenderAddress()`

Backwards Compatibility

This ERC does not change the consensus layer, so there are no backwards compatibility issues for Ethereum as a whole. Unfortunately it is not easily compatible with pre-ERC-4337 Smart Contract Accounts, because those Accounts do not have a `validateUserOp` function. If the Smart Contract Account has a function for authorizing a trusted `UserOperation` submitter, then this could be fixed by creating an ERC-4337 compatible Account that re-implements the verification logic as a wrapper and setting it to be the original Account's trusted `UserOperation` submitter.

Security Considerations

The `EntryPoint` contract will need to be audited and formally verified, because it will serve as a central trust point for *all* [ERC-4337]. In total, this architecture reduces auditing and formal verification load for the ecosystem, because the amount of work that individual *accounts* have to do becomes much smaller (they need only verify the `validateUserOp` function and its "check signature and pay fees" logic) and check that other functions are `msg.sender == ENTRY_POINT` gated (perhaps also allowing `msg.sender == self`), but it is nevertheless the case that this is done precisely by concentrating security risk in the `EntryPoint` contract that needs to be verified to be very robust.

Verification would need to cover two primary claims (not including claims needed to protect paymasters, and claims needed to establish p2p-level DoS resistance):

- **Safety against arbitrary hijacking**: The `EntryPoint` only calls to the `sender` with `userOp.calldata` and only if `validateUserOp` to that specific `sender` has passed.
- **Safety against fee draining**: If the `EntryPoint` calls `validateUserOp` and passes, it also must make the generic call with calldata equal to `userOp.calldata`

Copyright

Copyright and related rights waived via CC0.

Citation

Please cite this document as:

Vitalik Buterin (@vbuterin), Yoav Weiss (@yoavw), Dror Tirosh (@drortirosh), Shahaf Nacson (@shahafn), Alex Forshtat (@forshtat), Kristof Gazso (@kristofgazso), Tjaden Hess (@tjade273), "ERC-4337: Account Abstraction Using Alt Mempool," *Ethereum Improvement Proposals*, no. 4337, September 2021. Available: https://eips.ethereum.org/EIPS/eip-4337.